Git – das Kartenspiel*

Philipp Müller

8. Mai 2026

Dieses Spiel parodiert die agile Software-Entwicklung. Die Spieler bilden dabei ein Team von Entwicklern, die gemeinsam eine Software weiterentwickeln und dazu das Versionsverwaltungsprogramm Git verwenden. Ziel des Spiels ist es jedoch, möglichst viele der eigenen Quellcodezeilen durchzubringen und die anderen Spieler genau daran zu hindern.

1 – Aufbau

Das Spiel besteht aus drei Arten von Karten mit unterschiedlichen Rückseiten: **Spielermarker**- und **Issue**-Karten sowie die eigentlichen **Spielkarten**.
Die 9 Issue-Karten werden am Anfang von den anderen Karten getrennt, gut gemischt und bilden den Issue-Stapel. Eine Issue-Karte wird gezogen und aufgedeckt. Sie gibt das Ziel der ersten Spielrunde an und beginnt die Reihe der ausgelegten Karten, also das **Remote Repository** von Git.
Die Spielermarker werden von den restlichen Karten getrennt und an die jeweiligen Spieler verteilt.
Alle anderen Karten werden gut gemischt und bildet den Zugstapel. Jeder Spieler erhält davon drei auf die Hand, die sog. Handkarten. Diese entsprechen dem lokalen Git-Arbeitsverzeichnis des Spielers.

2 – Ablauf

Das Spiel wird in Runden, sog. Sprints, gespielt, die jeweils der Entwicklung einer Version der Software entsprechen. Sie beginnen mit dem Aufdecken einer **Issue**-Karte, die das Punkteziel des Sprints angibt, und enden mit der Auslieferung einer neuen Version der Software durch das Ausspielen einer **Release**-Karte. Dazwischen spielen alle Spieler reihum immer genau eine Handkarte aus und sagen dabei das, was auf der Karte in Anführungszeichen („. . .“) steht. Wenn die Karte am Remote Repository angelegt wird und Punkte einbringt (große Zahl in einem Kreis), dann muss sie mit einer Spielermarker-Karte markiert werden. In jedem Fall muss die zusätzliche Beschreibung auf der Karte ausgeführt werden, falls vorhanden.
Bestimmte Karten dürfen nur unter bestimmen Bedingungen ausgespielt werden. Falls jemand an die Reihe kommt und keine seiner Handkarten ausspielen kann, legt er sie alle auf den Abwurfstapel und zieht drei neue. Er spielt in dieser Runde also keine Karte aus.
Am Ende des Zuges **stockt** man seine Handkarten wieder auf (s. u. 2.1) und dann ist der nächste Spieler an der Reihe.
Ein Sprint ist beendet, sobald jemand eine Release-Karte ausspielt. Dann wird die aktuelle Runde **ausgewertet** (s. u. 2.2) und danach beginnt der nächste Sprint mit dem Aufdecken der nächsten Issue-Karte oder das Spiel ist ganz beendet (s. u. 3).

*Dies ist Version 1.1. Die aktuelle Version ist zu finden unter: `https://github.com/arkonia/GitGame`

2.1 – Aufstocken

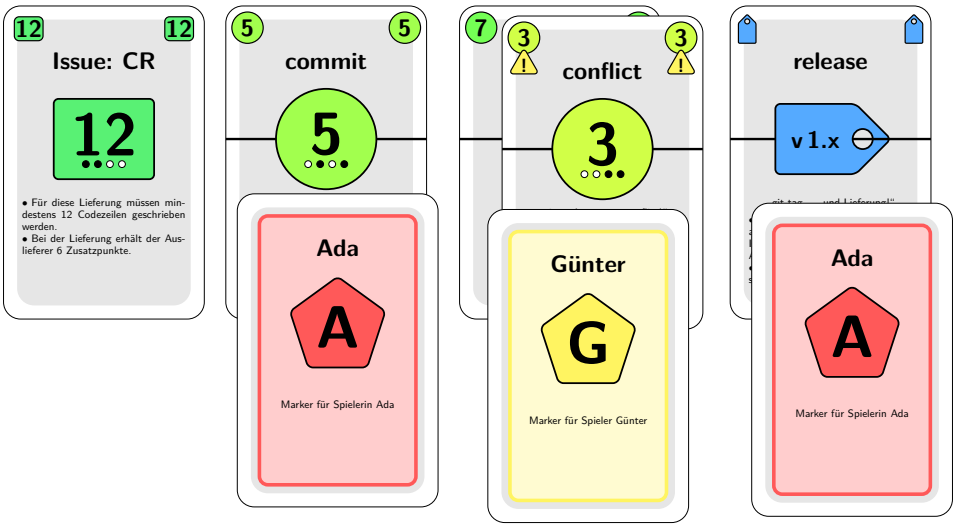
Falls man am Ende seines Zuges weniger als drei Handkarten hat, zieht man neue Karten vom Zugstapel bis man wieder drei auf der Hand hat.

2.2 – Auswerten

Sobald jemand eine Release-Karte ausspielt, wird eine neue Version der Software geliefert. D. h. der aktuelle Sprint wird ausgewertet. Jeder Spieler erhält Punkte entsprechend der Codezeilen all seiner Commits im Remote Repository, also zwischen Issue und Lieferung. Der Spieler der Release-Karte erhält zusätzliche Punkte wie auf der Issue-Karte angegeben.
Nach der Auswertung werden alle ausgelegten Karten und alle Handkarten abgeworfen. Jetzt können die Karten neu gemischt werden. Danach wird eine neue Issue-Karte ausgelegt und jeder Spieler erhält wieder drei Handkarten. Der Spieler nach dem Spieler, der die Release-Karte ausgespielt hat, beginnt dann den neuen Sprint.

Beispiel für eine Auswertung

Am Ende eines (recht kurzen) Sprints kann das Remote Repository z. B. folgendermaßen aussehen:



- Spieler A erhält 11 Punkte: Zahlkarte 5 plus 6 Extrapunkte fürs Release.
- Spieler D erhält 0 Punkte: Seine Zahlkarte 7 wurde von Spieler G geklaut (Merge Conflict)!
- Spieler G erhält 10 Punkte: Zahlkarte 7 und Zahlkarte 3.

Die Punkte können dann in einer kleinen Tabelle aufgeschrieben werden:

Runde	A	B	C
v. 1	11	0	10
v. 2			
v. 3			

3 – Ende

Am Anfang sollten alle Teilnehmer gemeinsam festlegen, wann das Spiel beendet ist (Definition of Done), z. B. nach 5 Lieferungen oder sobald ein Spieler mindestens 100 Punkte erreicht hat.
Gewonnen hat in jedem Fall selbstverständlich der Spieler mit der höchsten Gesamtpunktezahl.

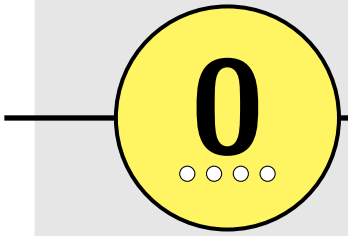
4 – Druckvorlagen

Die folgenden Seiten sind zum Ausdrucken und Ausschneiden vorgesehen. Zum Drucken wird dickes Papier empfohlen, mindestens 160 g-Papier. Die Karten sollten natürlich beidseitig ausgedruckt werden. Die Ränder links und rechts sind gleich breit, so dass Vorder- und Rückseite immer aufeinander treffen. Die Karten sind 59 mm × 92 mm groß.
Alternativ gibt es auch Dateien, die jeweils nur genau eine Karte je Seite zeigen:

`gitgame-de_Vorderseiten.pdf` und `gitgame-de_Rueckseite.pdf`

Diese haben einen zusätzlichen umlaufenden Rand von 3 mm, die Dateien sind also 65 mm × 98 mm groß.

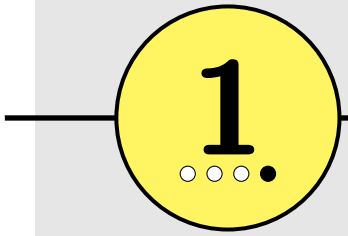
refactoring



„git push – refactoring!“

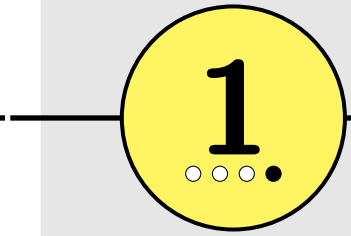
- Mit diesem Commit übernimmst du die vorherigen drei Commits als deine eigenen. Dazu müssen schon mindestens drei Commits ausliegen. Schieb sie unter diese Karte, alle diese Commits gelten ab jetzt als einer.

commit



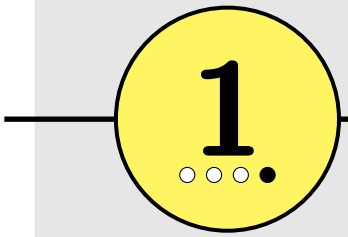
„git push!“

commit



„git push!“

conflict

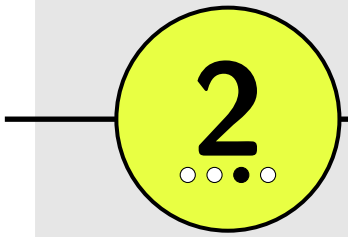


„git push – merge conflict!“

Mit diesem Commit verursachst und löst du einen Merge Conflict:

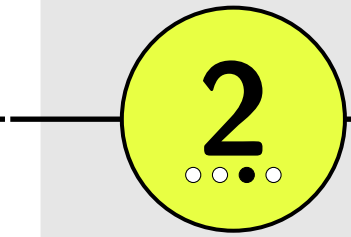
- Übernimm den vorherigen Commit als deinen eigenen. Schieb ihn unter diese Karte, diese beiden Commits gelten ab jetzt als einer.

commit



„git push!“

conflict

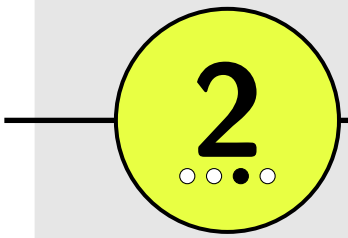


„git push – merge conflict!“

Mit diesem Commit verursachst und löst du einen Merge Conflict:

- Übernimm den vorherigen Commit als deinen eigenen. Schieb ihn unter diese Karte, diese beiden Commits gelten ab jetzt als einer.

request

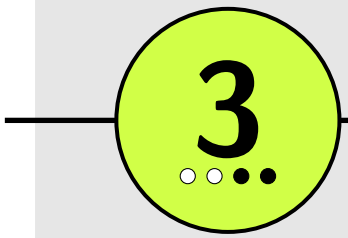


„git push – merge request!“

Mit diesem Commit *bittest* du deinen Kollegen um ein Code Review, dass er natürlich auch durchführt:

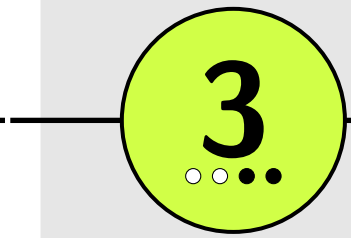
- Der nächste Spieler muss diese Karte mit einem Revert negieren oder einmal aussetzen.

commit



„git push!“

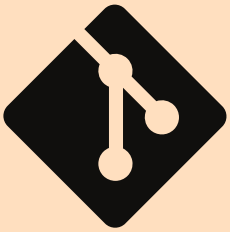
commit



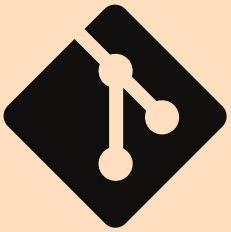
„git push!“



v. 1.1



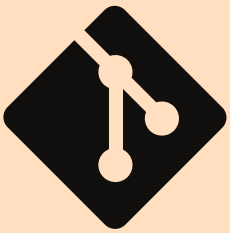
v. 1.1



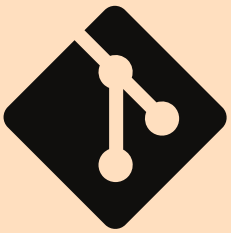
v. 1.1



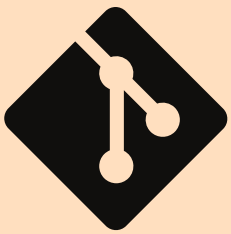
v. 1.1



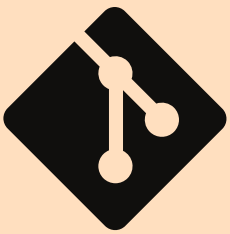
v. 1.1



v. 1.1



v. 1.1



v. 1.1



v. 1.1

conflict



„git push – merge conflict!“

Mit diesem Commit verursachst und löst du einen Merge Conflict:

- Übernimm den vorherigen Commit als deinen eigenen. Schieb ihn unter diese Karte, diese beiden Commits gelten ab jetzt als einer.

request



„git push – merge request!“

Mit diesem Commit *bittest* du deinen Kollegen um ein Code Review, dass er natürlich auch durchführt:

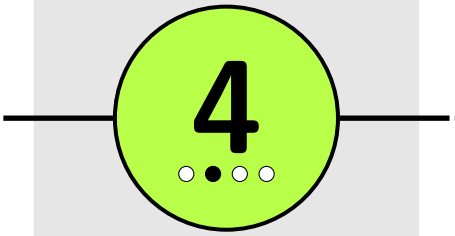
- Der nächste Spieler muss diese Karte mit einem Revert negieren oder einmal aussetzen.

commit



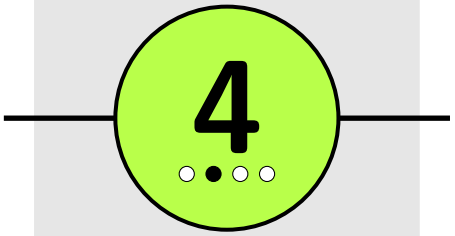
„git push!“

commit



„git push!“

conflict

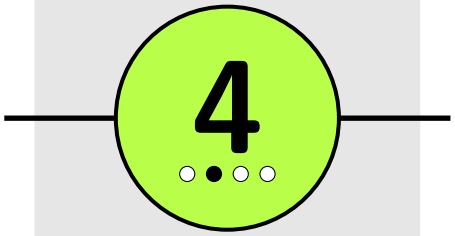


„git push – merge conflict!“

Mit diesem Commit verursachst und löst du einen Merge Conflict:

- Übernimm den vorherigen Commit als deinen eigenen. Schieb ihn unter diese Karte, diese beiden Commits gelten ab jetzt als einer.

request



„git push – merge request!“

Mit diesem Commit *bittest* du deinen Kollegen um ein Code Review, dass er natürlich auch durchführt:

- Der nächste Spieler muss diese Karte mit einem Revert negieren oder einmal aussetzen.

commit



„git push!“

commit



„git push!“

request



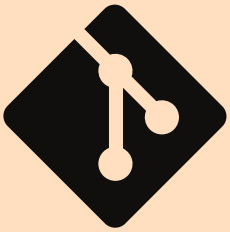
„git push – merge request!“

Mit diesem Commit *bittest* du deinen Kollegen um ein Code Review, dass er natürlich auch durchführt:

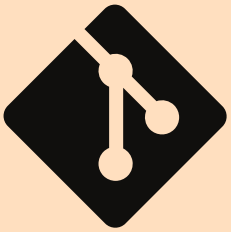
- Der nächste Spieler muss diese Karte mit einem Revert negieren oder einmal aussetzen.



v. 1.1



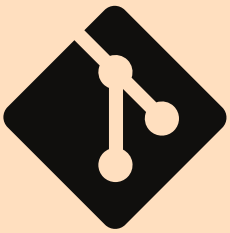
v. 1.1



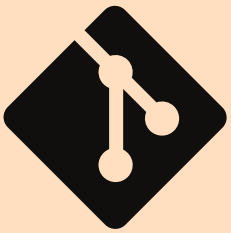
v. 1.1



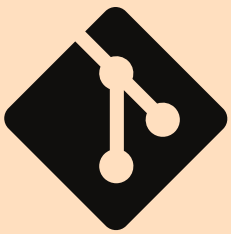
v. 1.1



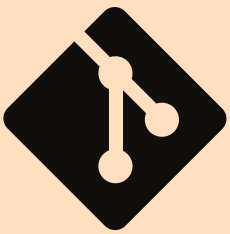
v. 1.1



v. 1.1



v. 1.1



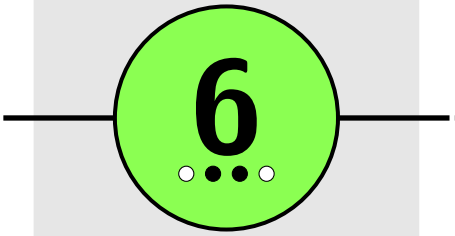
v. 1.1



v. 1.1

6 6

commit



„git push!“

6 6

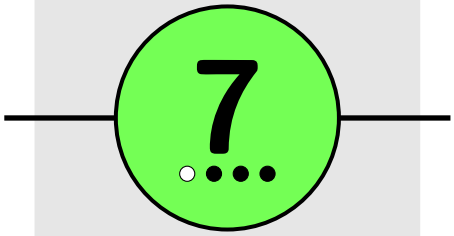
commit



„git push!“

7 7

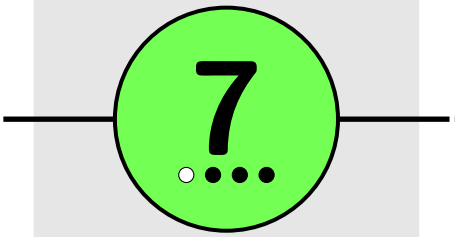
commit



„git push!“

7 7

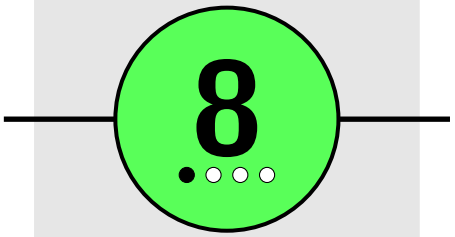
commit



„git push!“

8 8

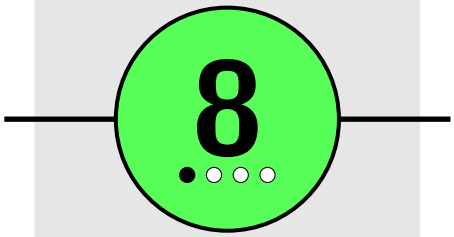
commit



„git push!“

8 8

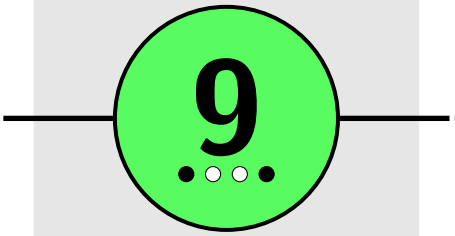
commit



„git push!“

9 9

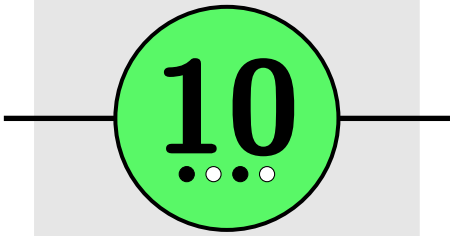
commit



„git push!“

10 10

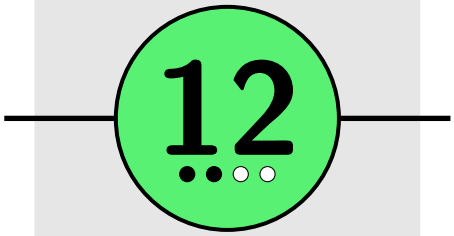
commit



„git push!“

12 12

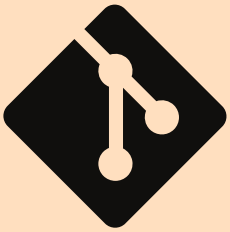
commit



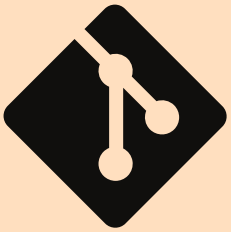
„git push!“



v. 1.1



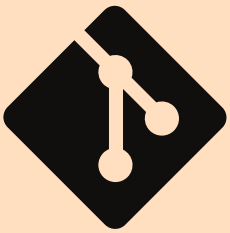
v. 1.1



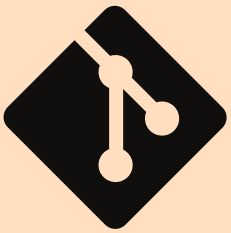
v. 1.1



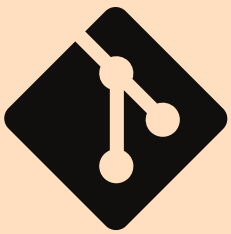
v. 1.1



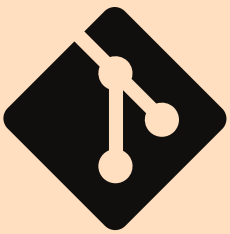
v. 1.1



v. 1.1



v. 1.1

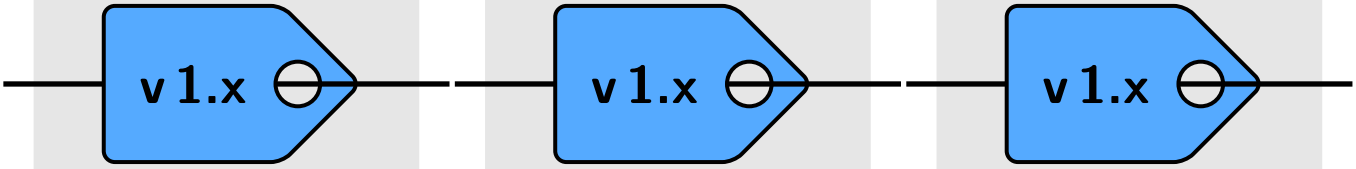


v. 1.1



v. 1.1

release



„git tag . . . und Lieferung!“

- Werte alle Karten im Repository aus und leg sie sowie die aktuellen Issues und alle Handkarten auf den Abwurfstapel.
- Leg diese Karte auf den Abwurfstapel und ziehe einen neuen CR.

release

„git tag . . . und Lieferung!“

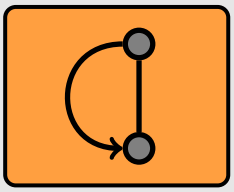
- Werte alle Karten im Repository aus und leg sie sowie die aktuellen Issues und alle Handkarten auf den Abwurfstapel.
- Leg diese Karte auf den Abwurfstapel und ziehe einen neuen CR.

release

„git tag . . . und Lieferung!“

- Werte alle Karten im Repository aus und leg sie sowie die aktuellen Issues und alle Handkarten auf den Abwurfstapel.
- Leg diese Karte auf den Abwurfstapel und ziehe einen neuen CR.

revert



„git revert!“

Der gerade geschriebene Code ist fehlerhaft und muss wieder weg:

- Muss unmittelbar auf einen Commit folgen und negiert ihn.
- Leg diese Karte und den Commit auf den Abwurfstapel.

revert

„git revert!“

Der gerade geschriebene Code ist fehlerhaft und muss wieder weg:

- Muss unmittelbar auf einen Commit folgen und negiert ihn.
- Leg diese Karte und den Commit auf den Abwurfstapel.

revert

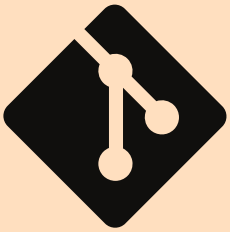
„git revert!“

Der gerade geschriebene Code ist fehlerhaft und muss wieder weg:

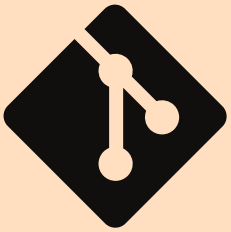
- Muss unmittelbar auf einen Commit folgen und negiert ihn.
- Leg diese Karte und den Commit auf den Abwurfstapel.



v. 1.1



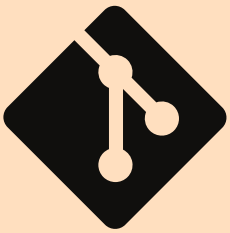
v. 1.1



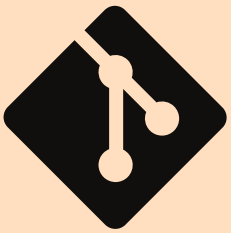
v. 1.1



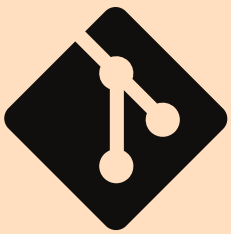
v. 1.1



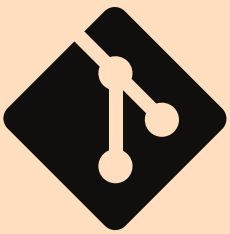
v. 1.1



v. 1.1



v. 1.1

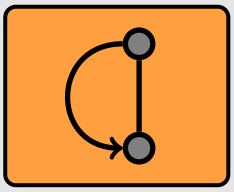


v. 1.1



v. 1.1

revert

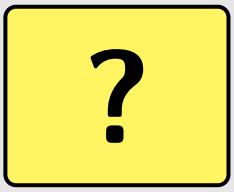


„git revert!“

Der gerade geschriebene Code ist fehlerhaft und muss wieder weg:

- Muss unmittelbar auf einen Commit folgen und negiert ihn.
- Leg diese Karte und den Commit auf den Abwurfstapel.

status

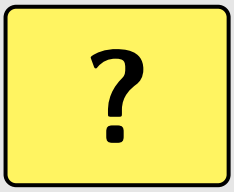


„git status!“

Du hast gerade keine Idee und brauchst erstmal einen Überblick:

- Leg diese Karte auf den Abwurfstapel.
- Zieh eine Handkarte zusätzlich, nachdem du **aufgestockt** hast.

status

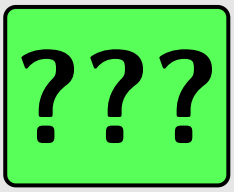


„git status!“

Du hast gerade keine Idee und brauchst erstmal einen Überblick:

- Leg diese Karte auf den Abwurfstapel.
- Zieh eine Handkarte zusätzlich, nachdem du **aufgestockt** hast.

help

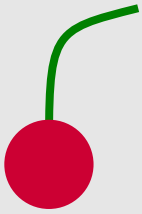


„git help!“

Du weißt gerade nicht weiter und brauchst Hilfe:

- Leg diese Karte auf den Abwurfstapel.
- Zieh drei Handkarten zusätzlich, nachdem du **aufgestockt** hast.

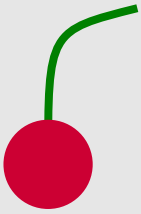
cherry-pick



„git cherry-pick!“

- **Stocke** zuerst deine Handkarten auf.
- Wähle dir danach bei einem beliebigen Spieler aus seinen Handkarten eine aus und füge sie deinen hinzu oder spiele sie sofort aus.

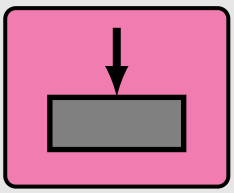
cherry-pick



„git cherry-pick!“

- **Stocke** zuerst deine Handkarten auf.
- Wähle dir danach bei einem beliebigen Spieler aus seinen Handkarten eine aus und füge sie deinen hinzu oder spiele sie sofort aus.

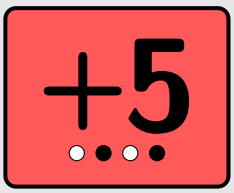
stash



„git stash!“

- Leg bis zu zwei Handkarten verdeckt vor Dir ab.
- Sie können weiterhin wie Handkarten ausgespielt werden, zählen aber nicht beim **Aufstocken** mit und gehen bei einem Release nicht verloren.

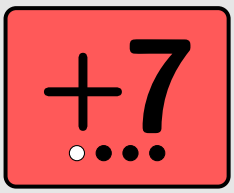
Bug



„Ich habe einen Fehler gefunden, der auch noch behoben werden muss.“

- Leg diese Karte zum aktuellen CR.
- Für die aktuelle Lieferung müssen noch 5 Codezeilen zusätzlich geschrieben werden.

Bug

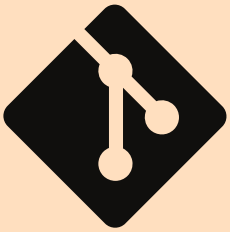


„Ich habe einen Fehler gefunden, der auch noch behoben werden muss.“

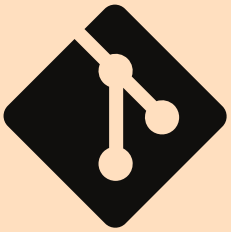
- Leg diese Karte zum aktuellen CR.
- Für die aktuelle Lieferung müssen noch 7 Codezeilen zusätzlich geschrieben werden.



v. 1.1



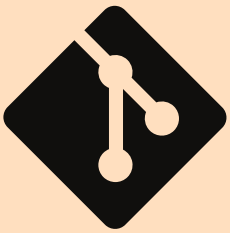
v. 1.1



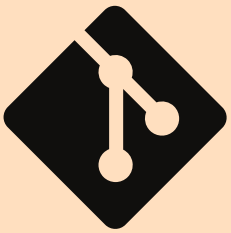
v. 1.1



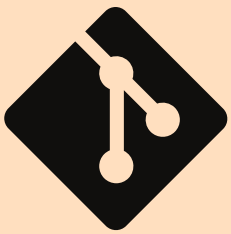
v. 1.1



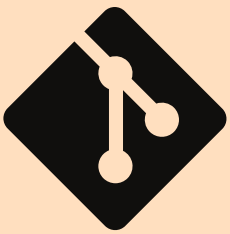
v. 1.1



v. 1.1



v. 1.1



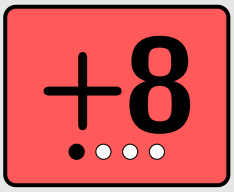
v. 1.1



v. 1.1

8 8

Bug

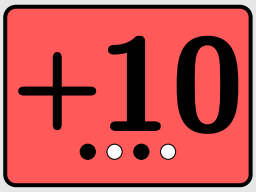


„Ich habe einen Fehler gefunden, der auch noch behoben werden muss.“

- Leg diese Karte zum aktuellen CR.
- Für die aktuelle Lieferung müssen noch 8 Codezeilen zusätzlich geschrieben werden.

10 10

Bug



„Ich habe einen Fehler gefunden, der auch noch behoben werden muss.“

- Leg diese Karte zum aktuellen CR.
- Für die aktuelle Lieferung müssen noch 10 Codezeilen zusätzlich geschrieben werden.

blame



„git blame!“

Du hast herausgefunden, wer den Fehler verursacht hat.

- Leg einen Marker eines Spielers deiner Wahl auf einen ausliegenden Bug. Dieser Spieler erhält den Wert des Bugs als Minuspunkte bei der Lieferung.

blame



„git blame!“

Du hast herausgefunden, wer den Fehler verursacht hat.

- Leg einen Marker eines Spielers deiner Wahl auf einen ausliegenden Bug. Dieser Spieler erhält den Wert des Bugs als Minuspunkte bei der Lieferung.

blame

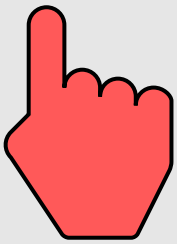


„git blame!“

Du hast herausgefunden, wer den Fehler verursacht hat.

- Leg einen Marker eines Spielers deiner Wahl auf einen ausliegenden Bug. Dieser Spieler erhält den Wert des Bugs als Minuspunkte bei der Lieferung.

blame



„git blame!“

Du hast herausgefunden, wer den Fehler verursacht hat.

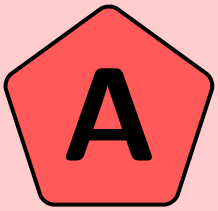
- Leg einen Marker eines Spielers deiner Wahl auf einen ausliegenden Bug. Dieser Spieler erhält den Wert des Bugs als Minuspunkte bei der Lieferung.

Ada



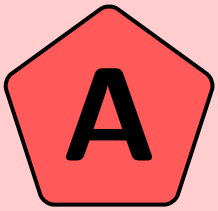
Marker für Spielerin Ada

Ada



Marker für Spielerin Ada

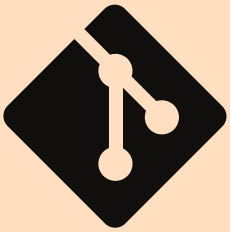
Ada



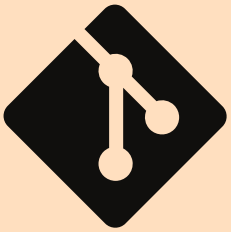
Marker für Spielerin Ada



v. 1.1



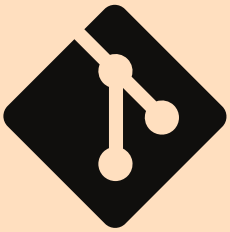
v. 1.1



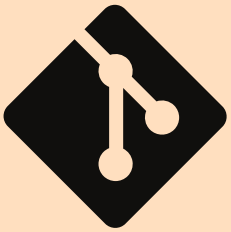
v. 1.1



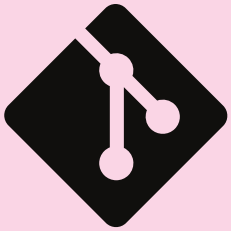
v. 1.1



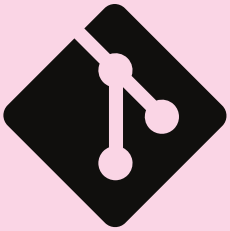
v. 1.1



v. 1.1



v. 1.1

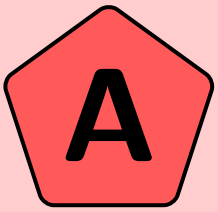


v. 1.1



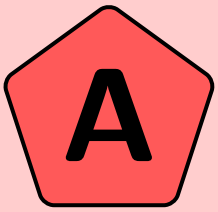
v. 1.1

Ada



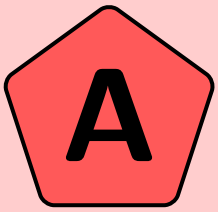
Marker für Spielerin Ada

Ada



Marker für Spielerin Ada

Ada



Marker für Spielerin Ada

Donald



Marker für Spieler Donald

Donald



Marker für Spieler Donald

Donald



Marker für Spieler Donald

Donald



Marker für Spieler Donald

Donald

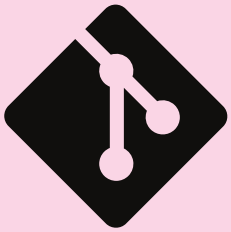


Marker für Spieler Donald

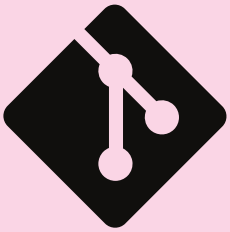
Donald



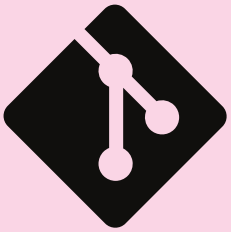
Marker für Spieler Donald



v. 1.1



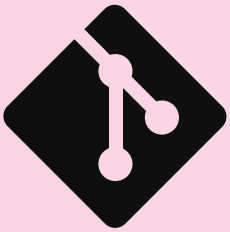
v. 1.1



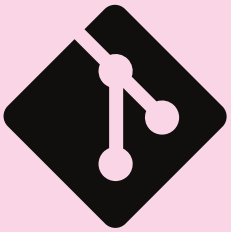
v. 1.1



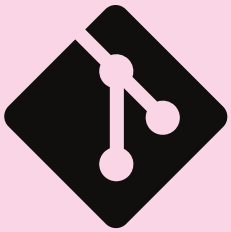
v. 1.1



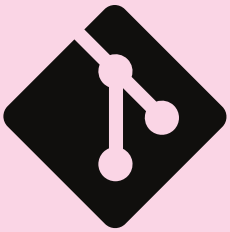
v. 1.1



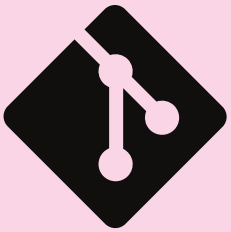
v. 1.1



v. 1.1



v. 1.1



v. 1.1

Günter



Marker für Spieler Günter

Günter



Marker für Spieler Günter

Günter



Marker für Spieler Günter

Günter



Marker für Spieler Günter

Günter



Marker für Spieler Günter

Günter



Marker für Spieler Günter

Konrad



Marker für Spieler Konrad

Konrad

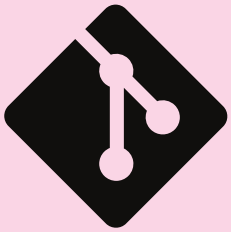


Marker für Spieler Konrad

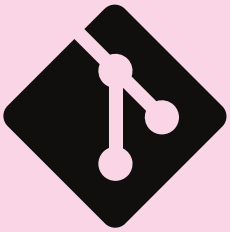
Konrad



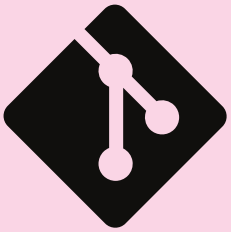
Marker für Spieler Konrad



v. 1.1



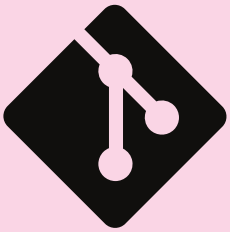
v. 1.1



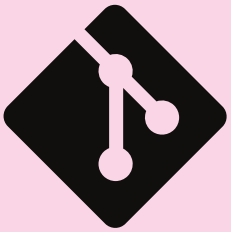
v. 1.1



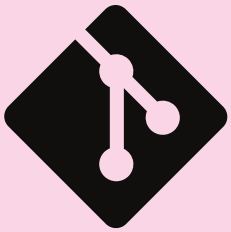
v. 1.1



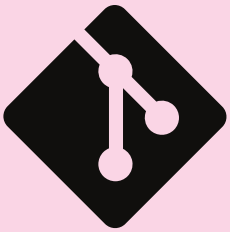
v. 1.1



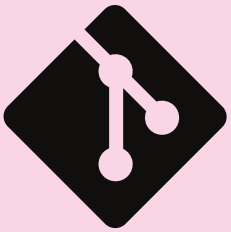
v. 1.1



v. 1.1



v. 1.1



v. 1.1

Konrad



Marker für Spieler Konrad

Konrad



Marker für Spieler Konrad

Konrad



Marker für Spieler Konrad

Linus



Marker für Spieler Linus

Linus



Marker für Spieler Linus

Linus



Marker für Spieler Linus

Linus



Marker für Spieler Linus

Linus

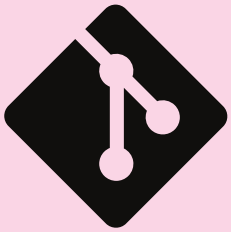


Marker für Spieler Linus

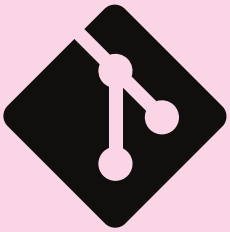
Linus



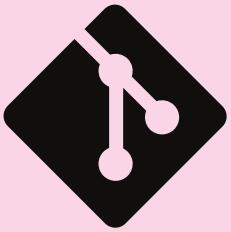
Marker für Spieler Linus



v. 1.1



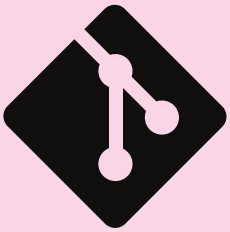
v. 1.1



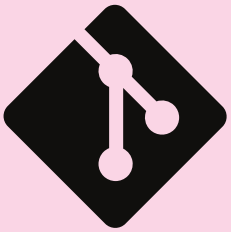
v. 1.1



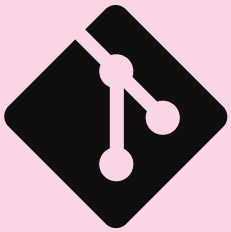
v. 1.1



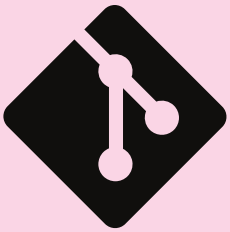
v. 1.1



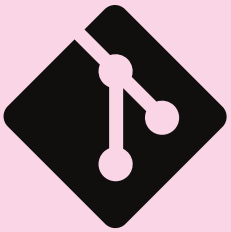
v. 1.1



v. 1.1



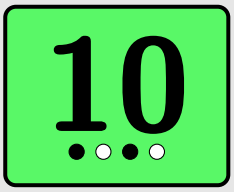
v. 1.1



v. 1.1

10 10

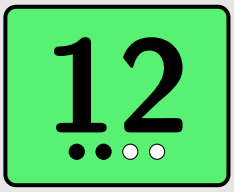
Issue: CR



- Für diese Lieferung müssen mindestens 10 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 5 Zusatzpunkte.

12 12

Issue: CR



- Für diese Lieferung müssen mindestens 12 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 6 Zusatzpunkte.

15 15

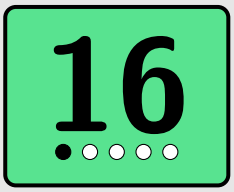
Issue: CR



- Für diese Lieferung müssen mindestens 15 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 7 Zusatzpunkte.

16 16

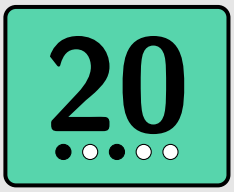
Issue: CR



- Für diese Lieferung müssen mindestens 16 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 8 Zusatzpunkte.

20 20

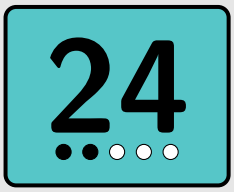
Issue: CR



- Für diese Lieferung müssen mindestens 20 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 10 Zusatzpunkte.

24 24

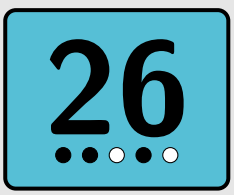
Issue: CR



- Für diese Lieferung müssen mindestens 24 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 12 Zusatzpunkte.

26 26

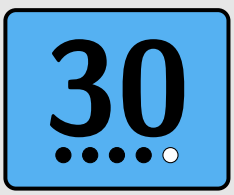
Issue: CR



- Für diese Lieferung müssen mindestens 26 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 13 Zusatzpunkte.

30 30

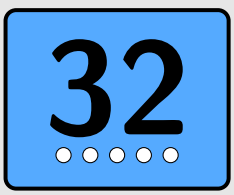
Issue: CR



- Für diese Lieferung müssen mindestens 30 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 15 Zusatzpunkte.

32 32

Issue: CR



- Für diese Lieferung müssen mindestens 32 Codezeilen geschrieben werden.
- Bei der Lieferung erhält der Auslieferer 16 Zusatzpunkte.



v. 1.1



v. 1.1



v. 1.1



v. 1.1



v. 1.1



v. 1.1



v. 1.1



v. 1.1



v. 1.1